



移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期主要工作

移位异或纠删码的高效实现研究

中期报告

学生：孙铎

导师：王强 副教授

哈尔滨工业大学（深圳）计算机科学与技术学院

2026 年 6 月 18 日



汇报提纲

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期主要工作

① 研究内容与进度

② 已完成工作与结果

③ 后期主要工作



研究内容与进度

存储系统需要可靠且低开销的冗余

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期主要工作

多副本：简单但占空间

- 数据需要冗余，才能在节点或磁盘失效后恢复
- 多副本直接保存多份数据，可靠但存储开销高

纠删码：用计算换空间

- 只保存少量校验块，也能恢复丢失数据
- 代价是写入、恢复和降级读都需要编解码计算

本课题的优化重点

让纠删码的编码、解码尽可能快，降低节省空间带来的计算代价。



研究内容与进度

为什么选择移位异或纠错码

移位异或纠错码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期主要工作

Reed-Solomon 纠错码

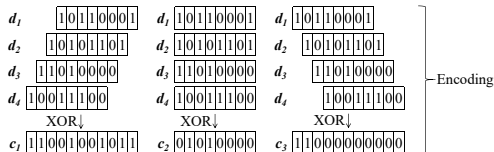
- 可靠、成熟，工程系统广泛使用
- 编解码需要有限域矩阵运算

移位异或纠错码

- 移位异或计算，复杂度优于 RS
- Two-tone 这一类中的 SOTA
- 缺少面向存储系统的高速实现
- RS 优化不能直接套用

因此：对 Two-tone 做系统级优化

例：4 个数据块经过不同偏移后异或生成 3 个校验块





研究内容与进度

中期结果：系统已落地，两个 SOTA 基线均有提升

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期主要工作

系统落地

- Ceph：知名开源分布式存储系统
- FastTT C++ 核心库 (约 2000 行)
- Ceph v18.2.7 纠删码插件
- IPDPS 2026 投稿

Ceph/ISA-L 基线

ISA-L: Ceph 官方生产级实现

+59.1%
编码平均提升

+50.9%
解码平均提升

解码最高提升 +182.1%

Cerasure 基线

学术 SOTA；未集成 Ceph，
按原环境测试

+13.2%
编码平均提升

+16.0%
解码平均提升

解码最高提升 +43.6%

孙铎

移位异或纠删码的高效实现研究

哈尔滨工业大学（深圳）计算机科学与技术学院

5 / 17



已完成工作与结果

Two-tone 如何编码和恢复

移位异或纠删码的
高效实现研究

孙铎

目录

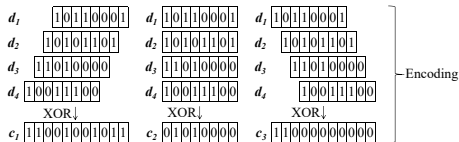
研究内容与进度

已完成工作与结果

后期主要工作

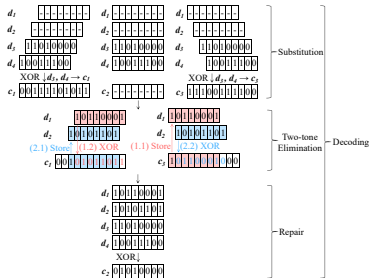
编码：移位后异或生成校验块

数据切成 k 个数据块，按不同偏移异或得到校验块。



恢复：代入校验关系恢复丢失块

存活块先代入校验关系，再按 Two-tone 顺序恢复丢失数据。



孙铎

移位异或纠删码的高效实现研究

哈尔滨工业大学（深圳）计算机科学与技术学院

6 / 17



已完成工作与结果

naive 实现的瓶颈主要在内存访问

移位异或纠错码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期主要工作

图片里能看到的问题

- 横向遍历：校验块分散写
- 纵向遍历：数据块反复读
- 长块访问：缓存局部性差

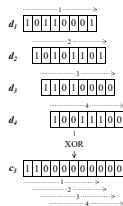
其他问题

- 解码后还要重编码修校验
- 多个算子可合并，naive 实现没有合并

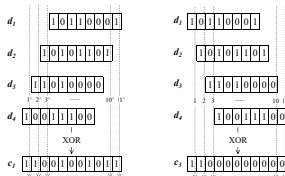
优化方向

提高访存局部性，减少访存次数。

naive 编码的两种访问模式



横向遍历：数据连续，校验分散写



纵向遍历：校验集中，数据反复读

孙铎

移位异或纠错码的高效实现研究

哈尔滨工业大学（深圳）计算机科学与技术学院

7 / 17



已完成工作与结果

FastTT 做了四个具体改动

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

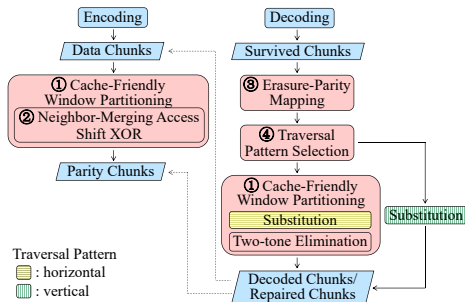
后期主要工作

四个改动

- ① 解出的数据块立刻去修校验块
- ② 长块切成短块窗口 (Tiling)
- ③ 相邻块先在寄存器合并再访存
- ④ 单块丢失走快速解码路径

落地结果

四项优化全部进入 FastTT, 并通过 Ceph 插件验证。



编码侧优化写入路径;
解码侧同时减少冗余流程和常见故障开销。

孙铎

移位异或纠删码的高效实现研究

哈尔滨工业大学 (深圳) 计算机科学与技术学院

8 / 17



已完成工作与结果

解出的数据块立刻去修校验块 (Erasure-Parity Mapping)

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

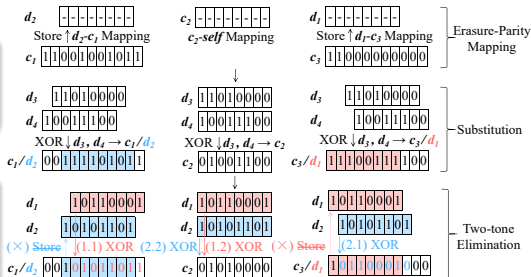
后期主要工作

原来的问题

先解码修复数据块，再重新编码修复
校验块：多跑了一遍。

FastTT 的做法

- 丢失数据块映射到存活校验块
- 丢失校验块映射到自身
- 图例： d_1 、 d_2 解出码字后立刻
用于重编码 c_2



孙铎

移位异或纠删码的高效实现研究

哈尔滨工业大学（深圳）计算机科学与技术学院

9 / 17



已完成工作与结果

长块切成短块窗口 (Cache-Friendly Window Partitioning)

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

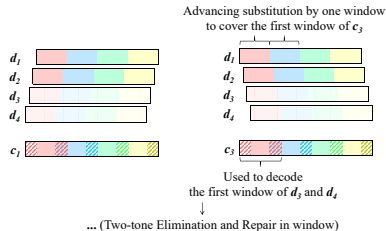
后期主要工作

原来的问题

对校验块来回跳转访问，数据还没用完就离开缓存。

FastTT 的做法

- 长块切成短块窗口
- 窗口内完成移位和异或
- 图例：同色片段在同一窗口内访问，编码解码都应用



孙铎

移位异或纠删码的高效实现研究

哈尔滨工业大学（深圳）计算机科学与技术学院

10 / 17



已完成工作与结果

相邻块先在寄存器合并再访存 (Neighbor-Merging Access)

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期主要工作

孙铎

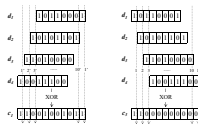
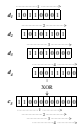
移位异或纠删码的高效实现研究

原来的问题

横向模式重复写校验块内存，
纵向模式重复读数据块内存。

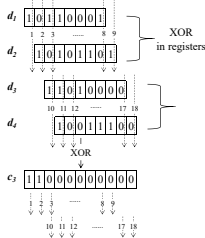
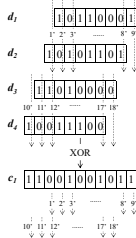
FastTT 的做法

- 数据块码字异或结果先存到向量寄存器，再连续写回校验块内存
- 注：具体实现中一个码字 256 位，等于 AVX2 向量宽度
- 写入校验块内存访问减少约一半，对数据块内存也没有重复读取



合并前横向：校验分散写

合并前纵向：数据反复读



合并后：相邻数据先合并，再连续写回校验块

哈尔滨工业大学（深圳）计算机科学与技术学院

11 / 17



已完成工作与结果

单块丢失走快速解码路径 (Traversal Pattern Selection)

移位异或纠错码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

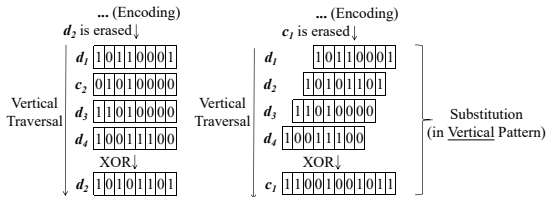
后期主要工作

原来的问题

仅单块丢失很常见，且解码单块计算简单（和编码单个块几乎一样），有很大优化潜力

FastTT 的做法

- 单块丢失选择纵向遍历
- 存活块异或一遍即可恢复
- 图例：缺失块一次异或得到



孙铎

移位异或纠错码的高效实现研究

哈尔滨工业大学（深圳）计算机科学与技术学院

12 / 17



已完成工作与结果

结果一：Ceph/ISA-L 生产基线下编码、解码均有提升

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期主要工作

Ceph v18.2.7 插件测试： 编码平均提升 +59.1%；解码平均提升 +50.9%；单块丢失解码最高提升 +182.1%。

配置 (k, m) 表示 k 个数据块、 m 个校验块；每个块 2MB，每组 2000 次。

编码吞吐 (GB/s)

配置	FastTT	ISA-L	Jerasure
(6,2)	34.74	24.37	17.03
(6,3)	31.69	17.59	7.64
(8,3)	31.35	17.03	7.08
(10,4)	17.59	13.56	4.58

四组配置全部超过 ISA-L；(10, 4) 仍保持领先。

少数解码低于 ISA-L：两块丢失需走通用解码；
(10, 4) 多块丢失受第 4 校验块的 2-symbol 位移影响。

解码吞吐 (GB/s)

配置	丢失	FastTT	ISA-L	Jerasure
(6,2)	1	95.99	38.38	42.70
(6,2)	2	25.60	24.89	16.26
(6,3)	1	95.18	39.43	44.57
(6,3)	2	24.24	24.77	12.12
(6,3)	3	19.18	18.16	5.39
(8,3)	1	93.11	33.01	37.04
(8,3)	2	26.18	23.16	10.97
(8,3)	3	21.44	18.38	5.00
(10,4)	1	81.01	34.42	35.11
(10,4)	2	20.96	22.23	10.62
(10,4)	3	15.26	17.08	4.73
(10,4)	4	11.53	13.97	3.69

孙铎

移位异或纠删码的高效实现研究

哈尔滨工业大学（深圳）计算机科学与技术学院

13 / 17



已完成工作与结果

结果二：Cerasure 学术基线下编码、解码均有提升

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期主要工作

Cerasure 原始环境测试： 编码平均提升 +13.2%；解码平均提升 +16.0%；解码最高提升 +43.6%。

Cerasure 未集成 Ceph，按原始测试方法对比；配置 (k, m) 表示 k 个数据块、 m 个校验块。

编码吞吐 (GB/s)

配置	FastTT	Cerasure
(6, 2)	47.97	44.20
(6, 3)	32.01	30.97
(8, 3)	32.75	28.72
(10, 4)	24.58	19.36

四组配置全部超过 Cerasure。

个别解码低于 Cerasure：两块丢失需走通用解码，无法使用单块直接异或快路径。

解码吞吐 (GB/s)

配置	丢失	FastTT	Cerasure
(6,2)	1	88.41	76.22
(6,2)	2	46.80	43.08
(6,3)	1	87.69	75.45
(6,3)	2	39.40	41.20
(6,3)	3	21.09	17.82
(8,3)	1	82.59	74.91
(8,3)	2	39.28	39.94
(8,3)	3	24.14	16.81
(10,4)	1	73.95	69.89
(10,4)	2	36.66	35.68
(10,4)	3	21.59	15.60
(10,4)	4	16.93	12.27

孙铎

哈尔滨工业大学（深圳）计算机科学与技术学院

移位异或纠删码的高效实现研究

14 / 17



已完成工作与结果

结果三：消融实验显示四个改动均有贡献

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期主要工作

(8, 3) 消融实验：完整 FastTT 相较原始实现，编码约 +149%，解码约 +367%。

表中 ✓ 表示启用该优化；吞吐单位为 GB/s。

单项增益	
优化	提升
边修校验	+100.2%
窗口划分	+43.0%
相邻合并	+58.6%
单块快路径	+43.5%

四个改动分别贡献解码流程、缓存局部性、写入次数和单块恢复路径。

组合吞吐 (GB/s)					
窗口	合并	边修校验	快路径	编码	解码
				13.16	7.54
		✓		13.16	16.85
✓		✓		23.32	24.10
✓	✓	✓		31.23	24.10
✓	✓	✓	✓	32.78	35.23

孙铎

哈尔滨工业大学（深圳）计算机科学与技术学院

移位异或纠删码的高效实现研究

15 / 17



后期主要工作

把 FastTT 讲扎实，再用它恢复 KV Cache

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期主要工作

1. 给优化方法补直接证据

- 现在已有端到端吞吐和消融结果
- 还要直接测：缓存命中率、实际访存次数
- 目标：直观说明省掉的内存开销

产出：补充实验图表，支撑论文修改和毕业论文。

2. 内存出问题时恢复 KV Cache

- 大模型推理会把中间结果存在键值缓存 (KV Cache) 里
- 节点内存故障后，重新预填充会让 GPU 再算一遍 KV，代价高
- 尝试用 FastTT 解码恢复丢失的 KV，减少重新计算

先调研 Mooncake：以 KV Cache 为中心的管理方案，再做恢复延迟原型。



Q&A

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期主要工作

感谢各位老师的耐心观看
请老师批评指正

Q&A